

Clase Práctica : Congestión

Tomás F. Melli

October 2025

Índice

1	Introducción	2
1.1	RFC 5681: TCP Congestion Control	2
1.2	Definiciones	2
1.3	La Ventana de Congestión (CWND)	3
1.4	Ventana del Emisor y Control de Flujo	3
2	Algoritmos de Control de Congestión en TCP	4
2.1	Slow Start	5
2.2	Ejercicio 1	6
2.3	Congestion Avoidance y Manejo de Time-out	6
2.4	TCP Tahoe	7
2.5	Fast Retransmit / Fast Recovery	8
2.6	TCP Reno	9
2.7	Consideraciones adicionales	10
3	Ejercicio 2	10
4	Ejercicio 3	12

1 Introducción

1.1 RFC 5681: TCP Congestion Control

Abstract

This document defines TCP's four intertwined congestion control algorithms: slow start, congestion avoidance, fast retransmit, and fast recovery. In addition, the document specifies how TCP should begin transmission after a relatively long idle period, as well as discussing various acknowledgment generation methods. This document obsoletes RFC 2581.

El RFC 5681 es un estándar fundamental para el funcionamiento de TCP en redes modernas. Describe cómo TCP gestiona la congestión de la red mediante cuatro algoritmos clave:

- **Slow Start:** Inicializa la ventana de congestión de manera exponencial hasta detectar signos de congestión. Permite que TCP descubra la capacidad de la red sin saturarla de inmediato.
- **Congestion Avoidance:** Una vez alcanzado un umbral, la ventana de congestión crece linealmente para evitar congestionar la red.
- **Fast Retransmit:** Detecta pérdidas de paquetes mediante la recepción de ACKs duplicados y retransmite el paquete perdido sin esperar al timeout.
- **Fast Recovery:** Ajusta la ventana de congestión tras un fast retransmit para mantener un flujo eficiente, evitando volver a slow start completamente.

Además, el RFC especifica cómo TCP debe reanudar la transmisión después de un período de inactividad prolongado y analiza distintos métodos de generación de acknowledgments, optimizando la eficiencia y la confiabilidad de la comunicación.

1.2 Definiciones

En TCP, el control de congestión se basa en varias variables y conceptos que permiten al emisor regular el flujo de datos hacia el receptor, detectando pérdidas y evitando saturar la red. A continuación se detallan las principales definiciones y su función:

- **Sender Maximum Segment Size (SMSS):** Define el tamaño máximo de segmento que el emisor puede enviar en la conexión TCP. Esto permite optimizar la transmisión evitando fragmentación y maximizando la eficiencia de la red.
- **Receiver Maximum Segment Size (RMSS):** Es el tamaño máximo de segmento que el receptor puede aceptar. El emisor debe respetar este límite para garantizar que los segmentos lleguen correctamente y se puedan ensamblar sin errores.
- **Full-Sized Segment:** Se refiere a un segmento que contiene la cantidad máxima de datos permitida por el SMSS. El envío de segmentos completos ayuda a utilizar de manera eficiente el ancho de banda disponible.
- **Receiver Window (RWND):** También llamada *ventana anunciada por el receptor*, indica cuántos bytes adicionales puede recibir el receptor sin desbordar su buffer. Es un mecanismo de control de flujo fundamental para evitar saturar al receptor.
- **Congestion Window (CWND):** Variable dinámica que limita la cantidad de datos que el emisor puede tener en tránsito (en vuelo) antes de recibir un ACK. CWND refleja el estado de la red: si hay congestión, CWND se reduce; si la red está libre, CWND aumenta.
- **Initial Window (IW):** Valor inicial de CWND después del *three-way handshake*. Determina cuántos bytes pueden enviarse al inicio de la conexión, influyendo directamente en el arranque de la transmisión.
- **Loss Window (LW):** Valor de CWND tras un timeout, que indica la reducción drástica de la ventana de congestión ante la pérdida de paquetes. Su función es prevenir que la red se sature mientras TCP se recupera.
- **Restart Window (RW):** Valor de CWND después de un período prolongado de inactividad (idle). Permite reanudar la transmisión de manera gradual, evitando un envío excesivo de datos tras la pausa.
- **Flight Size:** Cantidad de datos enviados pero aún no confirmados mediante ACKs, calculada como `LastByteSent - LastByteAcked`. Representa la "cantidad de datos en vuelo" y se usa para decidir cuánto más se puede enviar sin causar congestión.

- **Slow Start Threshold (SSTHRESH):** Umbral que determina cuándo TCP debe pasar de la fase de *Slow Start* a *Congestion Avoidance*. Durante *Slow Start*, CWND crece exponencialmente; tras superar SSTHRESH, el crecimiento se vuelve lineal para evitar congestionar la red.
- **Duplicate Acknowledgement (DupACK):** Se considera que un ACK es duplicado si se cumplen todas las siguientes condiciones:
 1. El receptor tiene datos sin confirmar.
 2. El ACK no contiene datos adicionales.
 3. No están activos los flags SYN ni FIN.
 4. El número de ACK es igual al máximo ACK previamente recibido.
 5. La *advertised window* es igual a la última recibida.

Los *DupACKs* son indicadores de posibles pérdidas de paquetes y activan mecanismos de retransmisión rápida en TCP.

En conjunto, estas variables permiten que TCP ajuste dinámicamente su comportamiento según las condiciones de la red, garantizando eficiencia, confiabilidad y prevención de congestión. La interacción entre CWND, RWND, IW, LW y RW define el flujo de datos y asegura que la comunicación sea segura y optimizada tanto para el emisor como para el receptor.

1.3 La Ventana de Congestión (CWND)

La **ventana de congestión** (CWND) es un concepto central en TCP y su control de congestión. Representa una estimación de la cantidad de información que el emisor puede introducir en la red sin afectar negativamente su rendimiento. Es decir, CWND limita la cantidad de datos "en vuelo" antes de recibir confirmaciones (*ACKs*) del receptor, permitiendo que TCP ajuste dinámicamente la velocidad de transmisión según las condiciones de la red.

Se definen las siguientes cantidades relacionadas:

- **MaxWindow:**

$$\text{MaxWindow} = \min(\text{RWND}, \text{CWND})$$

Esta expresión indica que la ventana máxima efectiva está limitada tanto por la capacidad del receptor (RWND) como por la estimación del emisor de la congestión de la red (CWND). En otras palabras, la cantidad de datos que puedo enviar está restringida por el menor de estos dos valores: la red o el host.

- **EffectiveWindow:**

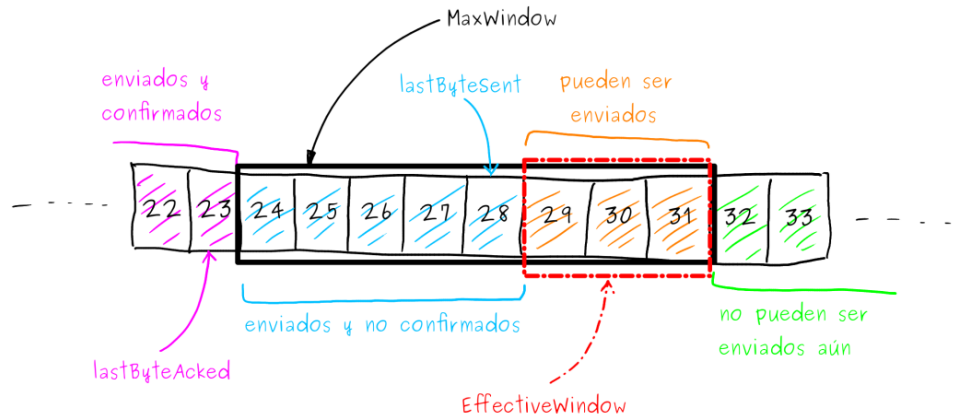
$$\text{EffectiveWindow} = \text{MaxWindow} - (\text{LastByteSent} - \text{LastByteAcked})$$

Representa la cantidad real de bytes que el emisor puede enviar en ese momento. Se calcula restando los datos que ya están en tránsito (es decir, los bytes enviados pero aún no confirmados) de la **MaxWindow**. Este valor es crucial para decidir cuánto más se puede despachar sin saturar la red.

En conjunto, estas definiciones permiten a TCP controlar el flujo de datos de manera eficiente, ajustando dinámicamente la transmisión y evitando congestionar la red. La interacción entre CWND, RWND, y la cantidad de datos ya enviados (**Flight Size**) asegura que la comunicación se mantenga estable y optimizada.

1.4 Ventana del Emisor y Control de Flujo

La **ventana del emisor** en TCP controla qué datos pueden ser enviados en un momento dado, combinando la estimación de congestión de la red (CWND) y la capacidad del receptor (RWND). Esta ventana determina tres categorías de paquetes: confirmados, en vuelo y permitidos para enviar.



En este escenario:

- **Paquetes 22 y 23:** Ya fueron enviados y confirmados por el receptor. Esto significa que están contabilizados en LastByteAcked y ya no forman parte de los datos en vuelo.
- **Paquetes 24 a 28:** Han sido enviados pero aún no han sido confirmados. Se encuentran en tránsito, y su cantidad contribuye al Flight Size, es decir, los bytes que están "en vuelo". En la fórmula de la ventana efectiva:

$$\text{EffectiveWindow} = \text{MaxWindow} - (\text{LastByteSent} - \text{LastByteAcked}),$$

estos paquetes restan capacidad disponible para nuevos envíos.

- **Paquetes 29, 30 y 31:** Pueden ser enviados inmediatamente. Esto se debe a que la EffectiveWindow es suficiente para acomodar estos tres paquetes sin superar la capacidad máxima de la ventana (MaxWindow).
- **Paquetes 32 y 33:** No pueden enviarse todavía. La ventana del emisor ya está llena hasta su límite máximo, definido por MaxWindow, y se debe esperar a la recepción de ACKs para avanzar.

En este escenario, la **MaxWindow** abarca los paquetes 24 a 31. Dentro de esta ventana, la **EffectiveWindow** determina cuántos paquetes adicionales se pueden despachar en un instante dado, teniendo en cuenta los datos ya enviados pero no confirmados.

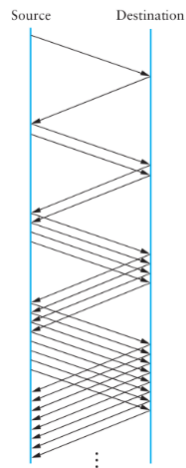
Este mecanismo garantiza que el emisor no sobresature la red ni al receptor, controlando dinámicamente el flujo de datos y optimizando la eficiencia de la comunicación.

2 Algoritmos de Control de Congestión en TCP

TCP utiliza varios algoritmos interrelacionados para controlar la congestión y mantener un flujo de datos eficiente. Los principales son:

- **Slow Start:** Permite iniciar la transmisión enviando pocos datos y aumentar gradualmente la ventana de congestión (CWND) a medida que se reciben ACKs. Este crecimiento exponencial evita saturar la red al inicio de la conexión.
- **Congestion Avoidance:** Una vez que la ventana de congestión alcanza un umbral denominado Ssthresh, el crecimiento de CWND se vuelve lineal. Este comportamiento evita congestionar la red, aumentando la ventana en aproximadamente un MSS por cada RTT.
- **Fast Retransmit / Fast Recovery:** Si se detecta la pérdida de un paquete mediante ACKs duplicados, TCP retransmite inmediatamente el segmento perdido sin esperar al timeout. Posteriormente, ajusta la CWND de manera controlada para recuperar la transmisión sin reiniciar completamente el Slow Start.

2.1 Slow Start



Inicialmente:

- CWND se inicializa con la Initial Window (IW), típicamente:

$$CWND = IW = 2 \times SMSS$$

- SSTHRESH se establece en un valor alto, por ejemplo la ventana máxima anunciada por el receptor:

$$SSTHRESH = \text{max advertised window}$$

Durante Slow Start:

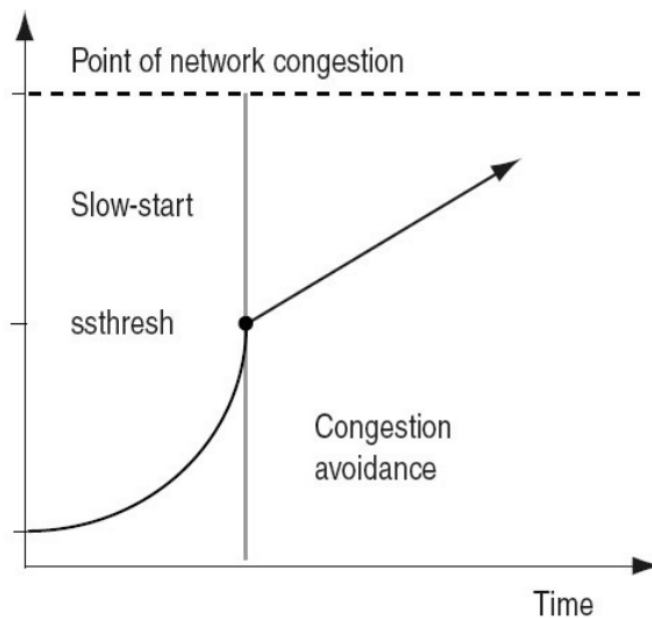
- Si $CWND < SSTHRESH$:

$$CWND += \min(N, SMSS) \quad \text{por cada ACK recibido}$$

donde N es la cantidad de bytes reconocidos por el ACK.

- La llegada de ACKs actúa como retroalimentación positiva, permitiendo que CWND crezca exponencialmente hasta alcanzar SSTHRESH.

Algoritmo: Slow Start + Congestion Avoidance



La transición de Slow Start a Congestion Avoidance se produce en el **punto SSTHRESH**, que indica el límite aproximado de capacidad de la red antes de que se produzca congestión. En el gráfico de CWND vs tiempo:

- Durante **Slow Start**, CWND crece exponencialmente con cada RTT, reflejando un aumento rápido del envío de datos.
- Al alcanzar **SSTHRESH**, comienza **Congestion Avoidance** y el crecimiento se vuelve lineal. Esto evita un aumento abrupto de la carga en la red, manteniendo un crecimiento más controlado y seguro.
- Si ocurre congestión (pérdida de paquetes o timeout), se ajusta **SSTHRESH** y CWND se reduce, normalmente a un valor cercano a 1-2 SMSS, iniciando un nuevo ciclo de Slow Start o Fast Recovery según la situación.

En conjunto, estos algoritmos permiten que TCP aumente la eficiencia de transmisión de manera adaptativa, equilibrando el crecimiento de la ventana con la capacidad de la red y minimizando la pérdida de paquetes.

2.2 Ejercicio 1

Considere el efecto de usar Slow Start en una conexión TCP recién establecida ($IW = 2 * SMSS$, $SSTHRESH = 64KB$, $SMSS = 2KB$), que tiene un RTT de 10 mseg y sin congestión ni errores presentes en la red. Si la RWND es de 24 KB, ¿Cuánto tiempo transcurre antes de que pueda ser enviada la primera ventana de recepción llena?

Resolución

La idea acá es simular el intercambio de mensajes teniendo en cuenta que el Ttx de un ráfaga es mucho más chico que el Tprop de la conexión ($Ttx \ll Tprop$). Esta es una condición fuerte pero asegura que los envíos se den de a ráfagas, lo cual simplifica la resolución del ejercicio, de manera de poder capturar la dinámica de intercambio de paquetes.

En el estado inicial, la conexión arranca con $CWND = IW = 2 * SMSS$. En el primer RTT se envía una ráfaga de dos segmentos llegando a un total de 4KB de datos en vuelo. Al momento de recibir cada ACK de vuelta, la ventana de congestión se incrementa como $CWND += \min(N, SMSS)$ (Cabe aclarar que en este caso $N=SMSS$). Esto deja la $CWND = 4 * SMSS = 8KB$.

Así, la próxima ráfaga será de 8KB y volverán 4 ACKs de 2KB cada uno, por lo que la ventana llegará a 16KB al comienzo del próximo envío, que sería el tercero. El próximo envío (el tercero), se envían 16KB y vuelven 8 ACKs de 2KB cada uno $\Rightarrow CWND=32KB$. A partir de acá, en el 4to RTT, el envío continúa con $FlightSize = \min(RWND, CWND) = \min(24, 32) = 24$, momento en el cual se dice que se envía la primera ventana de recepción llena.

Esto puede quedar más claro usando una tabla como la siguiente, donde en cada línea está el estado de las variables de la conexión que necesitamos para resolver el ejercicio:

RTT	CWND	RWND	FlightSize
1	4KB	24KB	4KB
2	8KB	24KB	8KB
3	16KB	24KB	16KB
4	32KB	24KB	24KB

2.3 Congestion Avoidance y Manejo de Time-out

Después de la fase de Slow Start, cuando la ventana de congestión (CWND) supera el umbral **SSTHRESH**, TCP entra en la fase de **Congestion Avoidance**.

Durante esta fase:

- Se busca aumentar la CWND de manera controlada, incrementando aproximadamente un SMSS por cada RTT.
- En términos de ACK recibidos, la actualización se realiza como:

$$CWND+ = SMSS \times \frac{SMSS}{CWND}$$

por cada ACK recibido. Este mecanismo corresponde a la estrategia de **Additive Increase**, que permite un crecimiento lineal de la ventana y evita congestionar la red abruptamente.

Manejo de Time-out

En caso de ocurrir un **time-out**:

- La ventana de congestión se reinicia con:

$$CWND = LW \text{ (1 SMSS)}$$

- El umbral de Slow Start se ajusta a:

$$SSTHRESH = \max\left(\frac{\text{FlightSize}}{2}, 2 \times \text{SMSS}\right)$$

- Se comienza de nuevo la fase de Slow Start.

El time-out actúa como una **retroalimentación negativa**, indicando que la red está saturada y que se debe reducir temporalmente la tasa de envío para evitar mayor congestión.

2.4 TCP Tahoe

TCP Tahoe es uno de los primeros algoritmos de control de congestión implementados en TCP. Su objetivo principal es evitar la congestión en la red mediante el ajuste dinámico de la *congestion window* (CWND). Tahoe combina tres mecanismos fundamentales: **Slow Start**, **Congestion Avoidance** y **Fast Retransmit/Reset en pérdida de paquetes**.

El algoritmo asume que la pérdida de paquetes es un indicador de congestión en la red. Cuando se detecta un timeout o pérdida de paquete, TCP Tahoe reinicia la ventana de congestión, reduciendo temporalmente la tasa de envío y permitiendo que la red se recupere.

Fases de TCP Tahoe

1. **Slow Start**: - TCP comienza con una CWND pequeña (por ejemplo, 1 o 2 SMSS). - Por cada ACK recibido, la CWND se incrementa exponencialmente:

$$CWND+ = N \quad (\text{bytes reconocidos por el ACK})$$

- Esto permite descubrir rápidamente la capacidad de la red sin saturarla al inicio.

2. **Congestion Avoidance (Additive Increase)**: - Una vez que CWND alcanza el SSTHRESH, el crecimiento deja de ser exponencial y pasa a lineal. - La CWND aumenta aproximadamente un SMSS por RTT:

$$CWND+ = \frac{\text{SMSS}^2}{CWND} \quad (\text{por cada ACK})$$

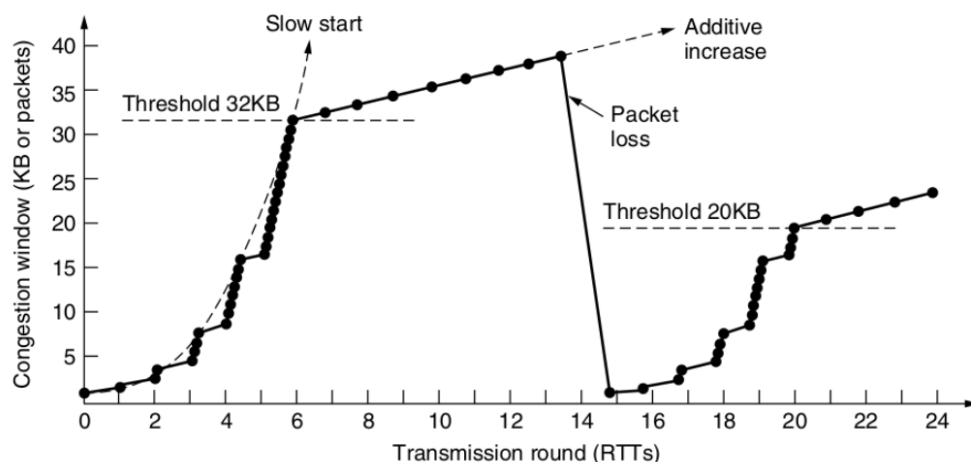
- Esta estrategia permite un aumento controlado de la ventana y reduce el riesgo de congestión.

3. **Manejo de pérdida de paquetes / Time-out**: - Ante un time-out o pérdida detectada:

$$CWND = 1 \text{ SMSS}, \quad SSTHRESH = \max(\text{FlightSize}/2, 2 \text{ SMSS})$$

- TCP Tahoe reinicia la fase de Slow Start con el nuevo threshold. - Esto representa la **retroalimentación negativa** ante congestión.

Gráfico

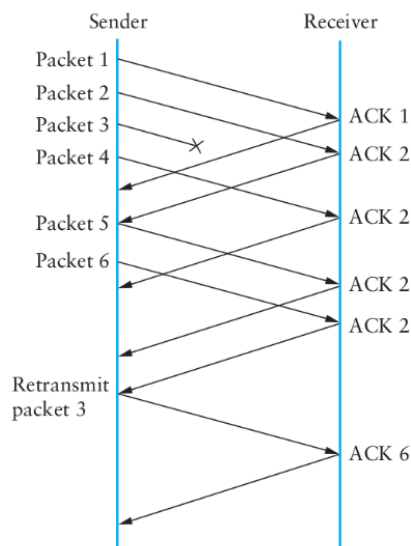


- **Slow Start hasta threshold 32 KB:** - La CWND crece exponencialmente hasta alcanzar 32 KB, mostrando un ascenso rápido.
- **Congestion Avoidance (Additive Increase):** - Después de 32 KB, la CWND crece de forma lineal por cada RTT, reflejando el control cuidadoso del aumento.
- **Pérdida de paquetes y reset:** - Ocurre un packet loss y la CWND cae abruptamente a 1 SMSS. - El threshold se ajusta a 20 KB, y TCP reinicia Slow Start hasta alcanzar nuevamente este nuevo límite.

En resumen, el ciclo de TCP Tahoe se repite: un crecimiento exponencial inicial (Slow Start), luego lineal (Congestion Avoidance), seguido de una caída brusca ante pérdida de paquetes, que fuerza un nuevo arranque con Slow Start hasta el nuevo threshold.

2.5 Fast Retransmit / Fast Recovery

TCP implementa los mecanismos de **Fast Retransmit** y **Fast Recovery** para acelerar la recuperación ante pérdida de paquetes sin esperar a un timeout completo.



Cuando el emisor recibe el **tercer ACK duplicado**, se asume que un segmento se ha perdido y debe ser retransmitido inmediatamente, sin esperar al time-out. Este ACK duplicado actúa como una **retroalimentación negativa** que indica congestión parcial en la red.

Fast Recovery

Tras detectar un paquete perdido mediante el tercer ACK duplicado, se aplican las siguientes reglas:

- Se ajusta el threshold de Slow Start:

$$SSTHRESH = \max(\text{FlightSize}/2, 2 * \text{SMSS})$$

- La ventana de congestión se inicializa como:

$$CWND = SSTHRESH + 3 * \text{SMSS}$$

- Mientras no se reconozcan nuevos datos, por cada ACK duplicado se incrementa:

$$CWND+ = \text{SMSS}$$

- La recuperación termina cuando llega el primer ACK que reconoce datos nuevos. En ese momento, la CWND se ajusta a:

$$CWND = SSTHRESH$$

y el flujo continúa con la fase de **Congestion Avoidance**.

Este mecanismo permite recuperar rápidamente los paquetes perdidos sin reiniciar completamente el ciclo de Slow Start, reduciendo la caída de throughput en la conexión.

2.6 TCP Reno

TCP Reno es una evolución de TCP Tahoe que mejora la eficiencia en la recuperación de pérdidas de paquetes mediante el uso de **Fast Retransmit** y **Fast Recovery**. A diferencia de Tahoe, que siempre reinicia la ventana de congestión a 1 SMSS tras un paquete perdido, Reno permite mantener parcialmente la ventana y continuar enviando datos durante la recuperación, evitando una caída drástica del throughput.

Fases principales de TCP Reno

1. **Slow Start:** - Al igual que Tahoe, Reno comienza con una CWND pequeña (1 o 2 SMSS). - La ventana crece exponencialmente por cada ACK recibido:

$$CWND+ = N \quad (\text{bytes reconocidos por el ACK})$$

- Esta fase continúa hasta alcanzar el umbral **SSTHRESH**.

2. **Congestion Avoidance (Additive Increase):** - Cuando CWND supera SSTHRESH, el crecimiento pasa a lineal:

$$CWND+ = \frac{SMSS^2}{CWND} \quad (\text{por cada ACK})$$

- Esto permite un aumento gradual de la ventana evitando congestionar la red abruptamente.

3. **Fast Retransmit / Fast Recovery:** - Al recibir el tercer ACK duplicado, el emisor retransmite el segmento perdido inmediatamente. - SSTHRESH se ajusta:

$$SSTHRESH = \max(\text{FlightSize}/2, 2 * SMSS)$$

- La CWND se inicializa como:

$$CWND = SSTHRESH + 3 * SMSS$$

- Por cada ACK duplicado recibido, se incrementa:

$$CWND+ = SMSS$$

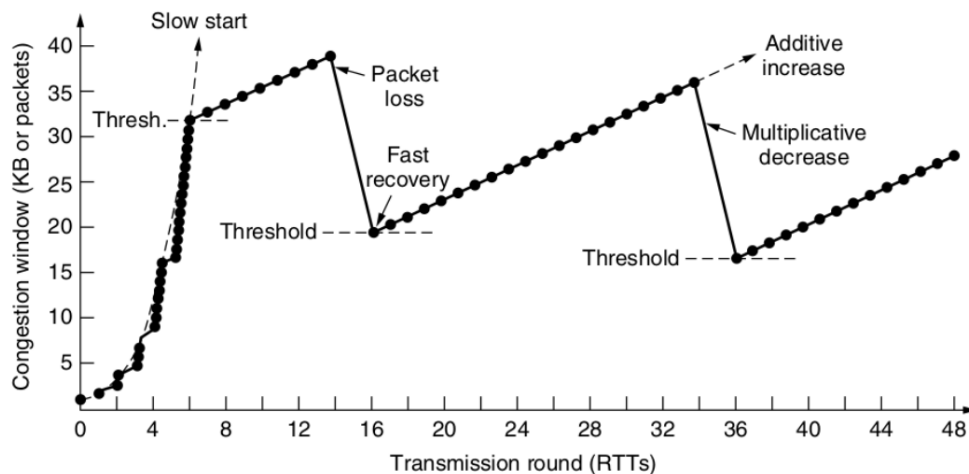
- Cuando llega el primer ACK que reconoce datos nuevos, CWND se ajusta a SSTHRESH y se continúa con Congestion Avoidance.

4. **Time-out:** - Si no llegan ACKs y ocurre un time-out, Reno se comporta como Tahoe:

$$CWND = 1 \text{ SMSS}, \quad SSTHRESH = \max(\text{FlightSize}/2, 2 * SMSS)$$

- Se reinicia la fase de Slow Start.

Gráfico



2.7 Consideraciones adicionales

Reiniciando conexiones idle

Cuando una conexión TCP permanece inactiva por un período prolongado, no recibe retroalimentación de la red. Esto puede ocurrir si no se envían datos, ya que los ACKs no llegan. Para manejar estas conexiones idle:

- Si no hay actividad por más de un RTT (Retransmission Timeout):

$$CWND = RW = \min(IW, CWND)$$

- Esto permite reiniciar la ventana de congestión de manera segura antes de reanudar el envío de datos.

Generando reconocimientos (ACKs)

TCP puede decidir retrasar la generación de ACKs para optimizar el tráfico de la red. Algunas consideraciones:

- Se puede esperar antes de enviar un ACK hasta:
 - Un máximo de 500 ms, o
 - Haber recibido 2 * MSS de bytes sin reconocer
- Esta estrategia permite agrupar ACKs y reducir overhead sin comprometer la confiabilidad.

3 Ejercicio 2

En una conexión recién establecida con $RTT = 200$ ms, el host receptor siempre anuncia una Advertised Window de 16 KB. La red está cargada al punto que si una ráfaga fuera de 16 KB o más, se perderían todos los segmentos de la misma.

1. ¿Cuánto vale la CWND luego de enviar un archivo de 40 KB?
2. 3 segundos después del envío del archivo, se envía otro archivo de 30 KB. ¿Cuánto tiempo tarda?

Solución

Como se mencionó previamente, una forma sencilla de encarar estos problemas es definir una tabla donde se lleva la cuenta de todas las variables que nos interesan para poder resolver el problema. Para esto, debemos decidir qué variables involucrar. Primero, el RTT es el elemento principal que lidera la evolución de la conexión. Segundo, las ventanas del control de flujo (RWND) y del control de congestión (CWND), cuyo mínimo define la máxima ventana disponible para enviar. En tercer lugar, también debemos llevar el SSTHRESH, dado que, ante la pérdida de una ráfaga, esta variable se actualizará para marcar el fin de Slow Start; esta variable comienza valiendo un valor arbitrariamente alto, por ejemplo 64 KB. Por último, llevamos cuenta de los datos en vuelo (FlightSize), dado que el control de congestión lo tiene en cuenta para ajustar la ventana, y de los datos enviados hasta el momento (LastByteSent o LBS), que marca cuánto hemos enviado con éxito hasta el momento.

RTT	CWND	RWND	SSTHRESH	FlightSize	LBS	Comentarios
1	4KB	16KB	64KB	4KB	4KB	La CWND inicia en $IW = 2 * SMSS$ y tomamos $SMSS = 2KB$. Siempre, $RWND = 16KB$ porque, en este caso, la Advertised Window nunca cambia.
2	8KB	16KB	64KB	8KB	12KB	Al principio aumenta con Slow Start, dado que el SSTHRESH es arbitrariamente alto.
3	16KB	16KB	64KB	16KB	28KB	Aca los datos están en vuelo, y no hay respuesta de ACKs dado que se pierden todos los segmentos de la ráfaga.
4	16KB	16KB	64KB	16KB	28KB	TIME OUT, dado que $RTO = 2 * RTT$.
5	2KB	16KB	8KB	2KB	14KB	La ventana se actualiza a $LW = 1 * SMSS = 2KB$ y el $SSTHRESH = \text{Max}(\text{FlightSize}/2, 2 * SMSS)$.
6	4KB	16KB	8KB	4KB	18KB	Aca sigue con Slow Start hasta que la CWND alcance el SSTHRESH.
7	8KB	16KB	8KB	8KB	26KB	A partir de aca se aumenta con Congestion Avoidance a razón de un SMSS por RTT.
8	10KB	16KB	8KB	10KB	36KB	Aca se aproxima el aumento de la CWND. Tomamos 1 SMSS por RTT.
9	12KB	16KB	8KB	4KB	40KB	Solo faltan 4KB para llegar a los 40KB.

Se puede decir que la CWND termina en 12KB luego de enviar todos los datos pero, la transmisión con éxito, termina luego de la llegada de los ACKs que faltan. Por aproximación tomamos que este último RTT, la CWND, aumentó otro SMSS y termina valiendo **14KB**.

Para el segundo inciso, la idea es acordarse que, luego de un RTO sin transmitir, la CWND, se reinicia a $RW = \min(IW, CWND) = 4KB$. Una vez tenido esto en cuenta, se puede ver cuanto tardaría en enviarse los próximos 30KB, usando la tabla:

RTT	CWND	RWND	SSTHRESH	FlightSize	LBS	Comentarios
1	4KB	16KB	8KB	4KB	4KB	Comienzo a contar en terminos de RTT desde 1 nuevamente. SSTHRESH se mantiene como venía hasta ahora porque no se aclara que deba cambiar.
2	8KB	16KB	8KB	8KB	12KB	Como $CWND < SSTHRESH$ aumento con Slow Start.
3	10KB	16KB	8KB	10KB	22KB	Aca comienzo Congestion Avoidance.
4	12KB	16KB	8KB	8KB	30KB	Al final de este RTT termina con éxito la transmisión.

Hicieron falta 4 RTTs para enviar los 30KB. La respuesta sería: $4 * 200ms = 0,8seg$

4 Ejercicio 3

Dada una conexión TCP recién establecida entre dos host para la cual el RTT es de 50ms. Los dos host están separados por un sólo router que también conecta otras redes y está cargado a tal punto que cada vez que una ráfaga de paquetes es de 20KB o más, se descartan todos los paquetes de la ráfaga. El host emisor tiene que enviar un archivo bastante grande que se está transmitiendo por horas y el host receptor siempre anuncia una *AdvertisedWindow* de 28KB.

a. Si se define que una conexión alcanza el *estado estacionario* en el momento que el Ssthresh converge a un valor a partir del cual ya no cambia más. ¿Cuánto tiempo tarda la conexión en alcanzar el estado estacionario? ¿Cuál es el valor del Ssthresh en dicho momento?

b. Finalizada la transferencia, se cierra la conexión, y se inicia una nueva en la que el host receptor siempre anuncia una *AdvertisedWindow* de 18KB. Si esta nueva conexión tuviera que transferir el mismo archivo, ¿tardaría más o menos tiempo que la anterior? (*Suponer las mismas condiciones de congestión en el router*)

Solución

RTT	CWND	RWND	Ssthresh	FlightSize	LBS	Comentarios
1	4	28	64	4	4	-
2	8	28	64	8	12	-
3	16	28	64	16	28	-
4	32	28	64	28	56	-
5	32	28	64	28	56	TIME OUT
6	2	28	14	2	30	-
7	4	28	14	4	34	-
8	8	28	14	8	42	-
9	16	28	14	16	58	-
10	18	28	14	18	76	-
11	20	28	14	20	96	-
12	20	28	14	20	96	TIME OUT
13	2	28	10	2	78	ACA EL Ssthresh ADOPTA EL ULTIMO VALOR Y NO CAMBIA MAS
14	4	28	10	4	82	-
15	8	28	10	8	90	-
16	12	28	10	12	102	-
17	14	28	10	14	116	-
18	16	28	10	16	132	-
19	18	28	10	18	150	-
20	20	28	10	20	170	-
21	20	28	10	20	170	TIME OUT
22	2	28	10	2	152	ACA EL Ssthresh YA DA LO MISMO

Bajo este diagrama la respuesta es, a partir del RTT 13 $\rightarrow 13 * 50ms = 650ms$

Para el segundo ítem, la MaxWindow del host emisor se mantendrá constante en 18KB regida por la RWND. El host receptor mantiene al emisor enviando datos justo por debajo del umbral de congestión. Esto hace que no se pierdan paquetes por congestión y que en cada RTT se envíen 18KB. En promedio, se enviarán más datos por RTT que en la conexión anterior, donde la red si llega a congestionarse y TCP ejecuta los mecanismos de recuperación que enlentecen el envío de datos.